

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service

- User Profile Service
- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer

- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service

- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)

- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)

- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service

- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance

- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js

- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP

- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)

- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards

- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native

- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker

- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)

- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:

- User authentication and authorization
- Data visualization and interaction
- Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)

- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization

- Data visualization and interaction
- Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction

- Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture

- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication

- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:

- Data Processing Service
- Notification Service
- User Profile Service
- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service

- Notification Service
- User Profile Service
- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service

- User Profile Service
- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer

- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service
 - Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)
- Technologies: NGINX or AWS API Gateway

1.3 Authentication & Authorization Service

- Manages user login, registration, and token issuance
- Implements OAuth 2.0 / JWT standards
- Technologies: Auth0, AWS Cognito, or custom OAuth server

1.4 Application Server Layer

- Microservices architecture
- Core services include:
 - Data Processing Service
 - Notification Service
 - User Profile Service

- Analytics Service
- Technologies: Node.js, Python (FastAPI), or Java Spring Boot

1.5 Data Storage

- Relational Database: PostgreSQL for structured data (user info, transactions)
- NoSQL Database: MongoDB or DynamoDB for unstructured data (logs, session data)
- Data Warehouse: Amazon Redshift or Snowflake for analytics and reporting

1.6 Message Queue & Event Bus

- Facilitates asynchronous communication
- Technologies: RabbitMQ, Apache Kafka, or AWS SNS/SQS

1.7 External Integrations

- Third-party APIs and services (payment gateways, external data sources)
- Managed via dedicated integration microservices

1.8 Infrastructure & Deployment

- Cloud provider: AWS, Azure, or GCP
- Containerization: Docker
- Orchestration: Kubernetes (EKS, AKS, GKE)
- CI/CD Pipeline: Jenkins, GitHub Actions, or GitLab CI

System Architecture Version 2

1. Components

1.1 User Interface (UI)

- Web-based frontend built with React.js
- Mobile app interface (iOS and Android) using React Native
- Responsibilities:
 - User authentication and authorization
 - Data visualization and interaction
 - Request submission and status monitoring

1.2 API Gateway

- Centralized API management layer
- Handles routing, load balancing, and security (SSL termination, rate limiting)